Compiler Design Lab

Date: 6-2-2026 **Name:** S Vishwajith
Assignment No.: 7 **Register No.:** 23BCE1145

Aim:

To write the given programs in LEX:

1. Write a LEX program to count the number of characters in a given input.
2. Write a LEX program to count the number of words in a sentence.
3. Write a LEX program to count the number of lines in a file.
4. Write a LEX program to count the number of vowels in the given input.
5. Write a LEX program to convert all lowercase letters to uppercase.
6. Write a LEX program to identify and count digits in the input.
7. Write a LEX program to identify and count keywords and identifiers.
8. Write a LEX program to recognize integers and floating-point numbers.
9. Write a LEX program to count the number of operators and operands.
10. Write a LEX program to remove single-line and multi-line comments from a C program.
11. Write a LEX program to recognize relational and logical operators.
12. Write a LEX program to count the number of uppercase and lowercase letters.
13. Write a LEX program to identify valid and invalid identifiers.
14. Write a LEX program to find and print all email IDs from a given text.
15. Write a LEX program to perform lexical analysis of a C program.
16. Write a LEX program to generate tokens for keywords, identifiers, constants, and operators.
17. Write a LEX program to implement a symbol table.
18. Write a LEX program to validate arithmetic expression.

Codes:

1.

```
/* Write a LEX program to count the number of characters in a given input. */


%{

#include <stdio.h>

int characters = 0;
```

```
%}

%%
.  {characters++;}
\n  {characters++;}
%%

int yywrap(){
   return 1;
}

int main(){
   printf("Input here: ");
   yylex();
   printf("The number of characters in the given input is: %d.\n", characters);
   return 0;
}
```

2.

```
/* Write a LEX program to count the number of words in a sentence. */

%{
   #include <stdio.h>
   int words = 0;
%}

%%
[a-zA-z]+   {words++;}
[ \n\t]+    {}
```

```
.       {}
%%

int yywrap(){
   return 1;
}

int main(){
   printf("Input here: ");
   yylex();
   printf("The number of words is: %d.\n", words);
   return 0;
}
```

3.

```
/* Write a LEX program to count the number of lines in a file. */

%{
   #include <stdio.h>
   int lines = 0;
%}

%%
.  {}
\n  {lines++;}
%%

int yywrap() {
   return 1;
```

```
}

int main() {
   printf("Input here: ");
   yylex();
   printf("The number of lines is: %d.\n", lines);
   return 0;
}
```

4.

```
/* Write a LEX program to count the number of vowels in the given input. */

%{
   #include <stdio.h>
   int vowels = 0;
%}

%%
[aeiouAEIOU]   {vowels++;}
.|\n          {}
%%

int yywrap() {
   return 1;
}

int main(){
   printf("Input here: ");
   yylex();
```

```
   printf("The number of vowels are: %d.\n", vowels);
   return 0;
}
```

5.

```
/* Write a LEX program to convert all lowercase letters to uppercase. */
%{
#include <stdio.h>
#include <ctype.h>
%}

%%
[a-z]     { yytext[0] = toupper(yytext[0]); ECHO; }
.|\n      { ECHO; }
%%

int yywrap() {
   return 1;
}

int main() {
   printf("Input here: ");
   fflush(stdout);
   yylex();
   return 0;
}
```

6.

```
/* Write a LEX program to identify and count digits in the input. */
```

```
%{
   #include <stdio.h>
   int digits = 0;
%}

%%
[0-9] {digits++;}
.|\n {}
%%

int yywrap() {
   return 1;
}

int main() {
   printf("Input here: ");
   yylex();
   printf("The number of digits is: %d.\n", digits);
   return 0;
}
```

7.

```
/* Write a LEX program to identify and count keywords and identifiers. */

%{
   #include <stdio.h>
   #include <string.h>
   int keywords = 0;
   int identifiers = 0;
```

```
  int is_keyword(char* token) {
    char *keywords[] = {"auto", "break", "case", "char", "const", "continue",
                "default", "do", "double", "else", "enum", "extern",
                "float", "for", "goto", "if", "int", "long", "register",
                "return", "short", "signed", "sizeof", "static", "struct",
                "switch", "typedef", "union", "unsigned", "void", "volatile",
                "while"};
    int n = sizeof(keywords) / sizeof(char*);
    for(int i = 0; i < n; i++) {
      if(strcmp(token, keywords[i]) == 0) {
        return 1;
      }
    }
    return 0;
  }
%}

identifier [a-zA-Z_][a-zA-Z0-9_]*
whitespace [ \t\n]+

%%
{identifier}   {
  if(is_keyword(yytext)) {
    keywords++;
    printf("KEYWORD: %s.\n", yytext);
  }
  else {
    identifiers++;
```

```
      printf("IDENTIFIER: %s.\n", yytext);
   }
}
{whitespace}   {}
.            {}
%%

int yywrap() {
   return 1;
}

int main() {
   printf("Write code here:\n");
   yylex();
   printf("The number of keywords is: %d.\n", keywords);
   printf("The number of identifiers is: %d.\n", identifiers);
}
```

8.

```
/* Write a LEX program to recognize integers and floating-point numbers. */

%{
   #include <stdio.h>
%}

%%
[0-9]+[.][0-9]+ {printf("Floating point value.\n");}
[0-9]+        {printf("Integer.\n");}
.|\n          {}
```

```
%%

int yywrap() {
   return 1;
}

int main() {
   printf("Input here:\n");
   yylex();
   return 0;
}
```

9.

```
/* Write a LEX program to count the number of operators and operands. */

%{
   #include <stdio.h>
   int operators = 0;
   int operands = 0;
%}

operator "<<"|">>"|"&&"|"||"|"<="|">="|"=="|"++"|"--"|"+"|"-
"|"*"|"/"|"%"|"<"|">"|"="|"&"|"|"|"^"
operand [a-zA-Z_][a-zA-Z0-9_]*
whitespace [ \t\n]+

%%
{operator}     {operators++;}
{operand}      {operands++;}
```

```
{whitespace}   {}
.            {}
%%

int yywrap() {
   return 1;
}

int main() {
   printf("Inputs here:\n");
   yylex();
   printf("Number of operands: %d.\n", operands);
   printf("Number of operators: %d.\n", operators);
   return 0;
}
```

10.

/* Write a LEX program to remove single-line and multi-line comments from a C program. */

```
%{
   #include <stdio.h>
   #include <string.h>
   #define MAX 100000
   char code[MAX];
   int x = 0;
   void add_text(const char *text) {
      int len = strlen(text);
      if (x + len < MAX) {
```

```
        strcpy(&code[x], text);
        x += len;
      }
   }
%}

%x SINGLE MULTI

%%
\"([^\\\"]|\\.)*\"  {add_text(yytext);}
\'([^\\\']|\\.)*\'  {add_text(yytext);}
"//"   {BEGIN(SINGLE);}
"/*"   {BEGIN(MULTI);}

<SINGLE>\n  {BEGIN(INITIAL); add_text("\n");}
<SINGLE>.   {}

<MULTI>"*/" {BEGIN(INITIAL);}
<MULTI>\n   {add_text("\n");}
<MULTI>.    {}

.|\n   {add_text(yytext);}
%%

int yywrap() {
   return 1;
}
```

```
int main() {
   printf("Enter C Code:\n");
   yylex();
   printf("The C code without any comments is:\n\n%s\n", code);
   return 0;
}
```

11.

```
/* Write a LEX program to recognize relational and logical operators. */

%{
   #include <stdio.h>
%}

relational "<="|">="|"=="|"!="|"<"|">"
logical "&&"|"||"|"!"

%%
{relational}   {printf("RELATIONAL OPERATOR: %s.\n", yytext);}
{logical}      {printf("LOGICAL OPERATOR: %s.\n", yytext);}
.|\n           {}
%%

int yywrap() {
   return 1;
}

int main() {
   printf("Inputs here:\n");
```

```
   yylex();
   return 0;
}
```

12.

```
/* Write a LEX program to count the number of uppercase and lowercase letters. */

%{
   #include <stdio.h>
   int upper = 0;
   int lower = 0;
%}

%%
[a-z]  {lower++;}
[A-Z]  {upper++;}
.|\n   {}
%%

int yywrap() {
   return 1;
}

int main() {
   printf("Input here:");
   yylex();
   printf("Number of uppercase letters is: %d.\n", upper);
   printf("Number of lowercase letters is: %d.\n", lower);
   return 0;
```

}

13.

/* Write a LEX program to identify valid and invalid identifiers. */

```
%{
  #include <stdio.h>
  #include <string.h>
  const char *keywords[] = {
    "int","float","char","double","if","else","while","for",
    "return","void","break","continue","switch","case","default",
    "do","struct","union","typedef","enum","const","unsigned",
    "signed","extern","static","sizeof","long","short","goto"
  };
  int isKeyword(const char *word) {
    int n = sizeof(keywords)/sizeof(keywords[0]);
    for(int i = 0; i < n; i++) {
      if(strcmp(word, keywords[i]) == 0)
        return 1;
    }
    return 0;
  }
%}

valid_id    [a-zA-Z_][a-zA-Z0-9_]*
invalid_id  [0-9][a-zA-Z0-9_]*

%%
{valid_id}{
```

```
   if(isKeyword(yytext))
      printf("%s : Keyword (not identifier)\n", yytext);
   else
      printf("%s : Valid Identifier\n", yytext);
}

{invalid_id}   {printf("%s : Invalid Identifier\n", yytext);}

[ \t\n]+       {}

.              {printf("%s : Invalid Identifier\n", yytext);}
%%

int yywrap() {
   return 1;
}

int main() {
   printf("Enter tokens:\n");
   yylex();
   return 0;
}
```

14.

```
/* Write a LEX program to find and print all email IDs from a given text. */

%{
   #include <stdio.h>
%}
```

```
email [a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}

%%
{email} {printf("Email found: %s\n", yytext);}

.|\n    {}
%%

int yywrap() {
   return 1;
}

int main() {
   printf("Inputs here:\n\n");
   yylex();
   return 0;
}
```

15.

```
/* Write a LEX program to perform lexical analysis of a C program. */

%{
   #include <stdio.h>
   #include <string.h>
%}

keyword    int|float|char|double|if|else|while|for|return|void|break|continue
id         [a-zA-Z_][a-zA-Z0-9_]*
```

```
number     [0-9]+(\.[0-9]+)?
operator   "=="|"!="|"<="|">="|"++"|"--"|"+"|"-"|"*"|"/"|"="|"<"|">"
delimiter  [;(),{}]

%%
{keyword}  {printf("Keyword: %s\n", yytext);}
{id}       {printf("Identifier: %s\n", yytext);}
{number}   {printf("Constant: %s\n", yytext);}
{operator} {printf("Operator: %s\n", yytext);}
{delimiter}{printf("Delimiter: %s\n", yytext);}
[ \t\n]+   {}
.          {printf("Unknown: %s\n", yytext);}
%%

int yywrap(){
   return 1;
}

int main(){
   printf("Enter the C code here:\n");
   yylex();
   return 0;
}
```

16.

/* Write a LEX program to generate tokens for keywords, identifiers, constants, and operators. */

```
%{
```

```
  #include <stdio.h>
%}

%%
int|float|char|double|if|else|while|for|return|void {printf("<KEYWORD, %s>\n", yytext);}
[a-zA-Z_][a-zA-Z0-9_]*                    {printf("<ID, %s>\n", yytext);}
[0-9]+(\.[0-9]+)?                         {printf("<CONST, %s>\n", yytext);}
"=="|"!="|"<="|">="|"+"|"-"|"*"|"/"|"="        {printf("<OP, %s>\n", yytext);}
[ \t\n]+                                  {}
.                                         {printf("<INVALID, %s>\n", yytext);}
%%

int yywrap(){
  return 1;
}

int main(){
  printf("Enter the C code here:\n");
  yylex();
  return 0;
}
```

17.

```
/* Write a LEX program to implement a symbol table. */

%{
  #include <stdio.h>
  #include <string.h>
  char symtab[100][50];
```

```
  int count = 0;
  int exists(char *s){
    for(int i=0;i<count;i++)
      if(strcmp(symtab[i],s)==0) return 1;
    return 0;
  }
%}

%%
int|float|char|double|if|else|while|for|return|void {}

[a-zA-Z_][a-zA-Z0-9_]* {
  if(!exists(yytext)){
    strcpy(symtab[count++], yytext);
    printf("Inserted into Symbol Table: %s\n", yytext);
  }
}

[ \t\n]+ ;
. ;
%%

int yywrap(){
  return 1;
}

int main(){
  printf("Enter the C code here:\n");
```

```
   yylex();
   printf("\nSymbol Table:\n");
   for(int i=0;i<count;i++)
      printf("%d. %s\n", i+1, symtab[i]);
   return 0;
}
```

18.

```
/* Write a LEX program to validate arithmetic expression */

%{
   #include <stdio.h>
   int valid = 1;
   int paren = 0;
%}

%%
[0-9]+              {}
[a-zA-Z_][a-zA-Z0-9_]*  {}
"("                 {paren++;}
")"                 {paren--;if(paren<0) valid=0;}
[+\-*/]             {}
[ \t\n]+            {}
.                   {valid = 0;}
%%

int yywrap(){
   return 1;
}
```

```
int main(){
  printf("Enter expression:\n");
  yylex();
  if(valid && paren==0)
    printf("Valid Arithmetic Expression\n");
  else
    printf("Invalid Arithmetic Expression\n");
  return 0;
}
```

Inputs and Outputs:

1.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q1.l; gcc lex.yy.c; ./a.exe

Input here: Hello World!

^Z

The number of characters in the given input is: 13.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q1.l; gcc lex.yy.c; ./a.exe

Input here: Hello World!
^Z
The number of characters in the given input is: 13.
* Terminal will be reused by tasks, press any key to close it.
```

2.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q2.l; gcc lex.yy.c; ./a.exe

Input here: Hello world, this is compiler assignment.

^Z

The number of words is: 6.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q2.l; gcc lex.yy.c; ./a.exe

Input here: Hello world, this is compiler assignment.
^Z
The number of words is: 6.
* Terminal will be reused by tasks, press any key to close it.
```

3.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q3.l; gcc lex.yy.c; ./a.exe

Input here: Hello

Bye

Compiler.

^Z

The number of lines is: 3.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q3.l; gcc lex.yy.c; ./a.exe

Input here: Hello
Bye
Compiler.
^Z
The number of lines is: 3.
* Terminal will be reused by tasks, press any key to close it.
```

4.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q4.l; gcc lex.yy.c; ./a.exe

Input here: Hello World!

^Z

The number of vowels are: 3.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q4.l; gcc lex.yy.c; ./a.exe

Input here: Hello World!
^Z
The number of vowels are: 3.
* Terminal will be reused by tasks, press any key to close it.
```

5.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q5.l; gcc lex.yy.c; ./a.exe

Input here: Hello world!

HELLO WORLD!

Bye.

BYE.

^Z

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q5.l; gcc lex.yy.c; ./a.exe

Input here: Hello world!
HELLO WORLD!
Bye.
BYE.
^Z
* Terminal will be reused by tasks, press any key to close it.
```

6.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q6.l; gcc lex.yy.c; ./a.exe

Input here: H3ll0 Fl4m3s!

^Z

The number of digits is: 4.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q6.l; gcc lex.yy.c; ./a.exe

Input here: H3ll0 Fl4m3s!
^Z
The number of digits is: 4.
* Terminal will be reused by tasks, press any key to close it.
```

7.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q7.l; gcc lex.yy.c; ./a.exe

Write code here:

#include <stdio.h>

IDENTIFIER: include.

IDENTIFIER: stdio.

IDENTIFIER: h.

int main(){

KEYWORD: int.

IDENTIFIER: main.

int a = 6 + 7;

KEYWORD: int.

IDENTIFIER: a.

return 0;

KEYWORD: return.

}

^Z

The number of keywords is: 3.

The number of identifiers is: 5.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q7.l; gcc lex.yy.c; ./a.exe

Write code here:
#include <stdio.h>
IDENTIFIER: include.
IDENTIFIER: stdio.
IDENTIFIER: h.
int main(){
KEYWORD: int.
IDENTIFIER: main.
        int a = 6 + 7;
KEYWORD: int.
IDENTIFIER: a.
        return 0;
KEYWORD: return.
}
^Z
The number of keywords is: 3.
The number of identifiers is: 5.
* Terminal will be reused by tasks, press any key to close it.
```

8.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q8.l; gcc lex.yy.c; ./a.exe

Input here:

11.11

Floating point value.

11

Integer.

^Z

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q8.l; gcc lex.yy.c; ./a.exe

Input here:
11.11
Floating point value.
11
Integer.
^Z
* Terminal will be reused by tasks, press any key to close it.
```

9.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q9.l; gcc lex.yy.c; ./a.exe

Inputs here:

a <<= b-- + ++c

^Z

Number of operands: 3.

Number of operators: 5.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q9.l; gcc lex.yy.c; ./a.exe

Inputs here:
a <<= b-- + ++c
^Z
Number of operands: 3.
Number of operators: 5.
* Terminal will be reused by tasks, press any key to close it.
```

10.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q10.l; gcc lex.yy.c; ./a.exe

Enter C Code:

#include <stdio.h>

```
//Single line comment.
/* Multi
line
comment.
*/
int main(){
    int a = 6 + 7;
    return 0;
}
^Z
```

The C code without any comments is:

```
#include <stdio.h>
int main(){
    int a = 6 + 7;
    return 0;
```

}

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q10.l; gcc lex.yy.c; ./a.exe

Enter C Code:
#include <stdio.h>
//Single line comment.
/* Multi
line
comment.
*/
int main(){
        int a = 6 + 7;
        return 0;
}
^Z
The C code without any comments is:

#include <stdio.h>





int main(){
        int a = 6 + 7;
        return 0;
}

* Terminal will be reused by tasks, press any key to close it.
```

11.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q11.l; gcc lex.yy.c; ./a.exe

Inputs here:

<

RELATIONAL OPERATOR: <.

<=

RELATIONAL OPERATOR: <=.

&&

LOGICAL OPERATOR: &&.

&

||

LOGICAL OPERATOR: ||.

^Z

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q11.l; gcc lex.yy.c; ./a.exe

Inputs here:
<
RELATIONAL OPERATOR: <.
<=
RELATIONAL OPERATOR: <=.
&&
LOGICAL OPERATOR: &&.
&
||
LOGICAL OPERATOR: ||.
^Z
* Terminal will be reused by tasks, press any key to close it.
```

12.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q12.l; gcc lex.yy.c; ./a.exe

Input here: Hello World!

^Z

Number of uppercase letters is: 2.

Number of lowercase letters is: 8.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q12.l; gcc lex.yy.c; ./a.exe

Input here: Hello World!
^Z
Number of uppercase letters is: 2.
Number of lowercase letters is: 8.
* Terminal will be reused by tasks, press any key to close it.
```

13.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q13.l; gcc lex.yy.c; ./a.exe

Enter tokens:

int sit

int : Keyword (not identifier)

sit : Valid Identifier

9ab-*bc

9ab : Invalid Identifier

- : Invalid Identifier

* : Invalid Identifier

bc : Valid Identifier

^Z

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q13.l; gcc lex.yy.c; ./a.exe

Enter tokens:
int sit
int : Keyword (not identifier)
sit : Valid Identifier
9ab-*bc
9ab : Invalid Identifier
- : Invalid Identifier
* : Invalid Identifier
bc : Valid Identifier
^Z
```

14.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q14.l; gcc lex.yy.c; ./a.exe

Inputs here:

ab

vishwajith.s2023@vitstudent.ac.in

Email found: vishwajith.s2023@vitstudent.ac.in

something@something.something

Email found: something@something.something

^Z

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q14.l; gcc lex.yy.c; ./a.exe

Inputs here:

ab
vishwajith.s2023@vitstudent.ac.in
Email found: vishwajith.s2023@vitstudent.ac.in
something@something.something
Email found: something@something.something
^Z
* Terminal will be reused by tasks, press any key to close it.
```

15.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q15.l; gcc lex.yy.c; ./a.exe

Enter the C code here:

#include <stdio.h>

Unknown: #

Identifier: include

Operator: <

Identifier: stdio

Unknown: .

Identifier: h

Operator: >

int main(){

Keyword: int

Identifier: main

Delimiter: (

Delimiter:)

Delimiter: {

    int a = 6+7;

Keyword: int

Identifier: a

Operator: =

Constant: 6

Operator: +

Constant: 7

Delimiter: ;

    return 0;

Keyword: return

Constant: 0

Delimiter: ;

}

Delimiter: }

^Z

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q15.l; gcc lex.yy.c; ./a.exe

Enter the C code here:
#include <stdio.h>
Unknown: #
Identifier: include
Operator: <
Identifier: stdio
Unknown: .
Identifier: h
Operator: >
int main(){
Keyword: int
Identifier: main
Delimiter: (
Delimiter: )
Delimiter: {
        int a = 6+7;
Keyword: int
Identifier: a
Operator: =
Constant: 6
Operator: +
Constant: 7
Delimiter: ;
        return 0;
Keyword: return
Constant: 0
Delimiter: ;
}
Delimiter: }
^Z
* Terminal will be reused by tasks, press any key to close it.
```

16.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q16.l; gcc lex.yy.c; ./a.exe

Enter the C code here:

int main(){

<KEYWORD, int>

<ID, main>

<INVALID, (>

<INVALID,)>

<INVALID, {>

return 0;

<KEYWORD, return>

<CONST, 0>

<INVALID, ;>

}

<INVALID, }>

^Z

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q16.l; gcc lex.yy.c; ./a.exe

Enter the C code here:
int main(){
<KEYWORD, int>
<ID, main>
<INVALID, (>
<INVALID, )>
<INVALID, {>
        return 0;
<KEYWORD, return>
<CONST, 0>
<INVALID, ;>
}
<INVALID, }>
^Z
* Terminal will be reused by tasks, press any key to close it.
```

17.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q17.l; gcc lex.yy.c; ./a.exe

Enter the C code here:

int main(){

Inserted into Symbol Table: main

return 0;

}

^Z

Symbol Table:

1. main

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q17.l; gcc lex.yy.c; ./a.exe

Enter the C code here:
int main(){
Inserted into Symbol Table: main
        return 0;
}
^Z

Symbol Table:
1. main
* Terminal will be reused by tasks, press any key to close it.
```

18.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q18.l; gcc lex.yy.c; ./a.exe

Enter expression:

a + (b - c) * d - ((e + f) / g)

^Z

Valid Arithmetic Expression

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q18.l; gcc lex.yy.c; ./a.exe

Enter expression:
a + (b - c) * d - ((e + f) / g)
^Z
Valid Arithmetic Expression
* Terminal will be reused by tasks, press any key to close it.
```

Result:

The LEX programs written for the given questions were successfully compiled and run.